

A Code Snippets of Key RTL Modules

This appendix collects representative code excerpts from the Verilog implementation of the GNN accelerator. Snippets focus on the computational kernels (parallel MAC neuron, ReLU/saturation, layer normalization, pipelined edge network), the top-level phase FSM, and the small primitive blocks (counter, multiplier).

A.1 Parallel MAC Neuron (neuron.v)

4-lane parallel multiply-accumulate; processes 4 weight×data pairs per cycle and accumulates with a 2-level adder tree. Cuts counter cycles by 4× vs the scalar version (e.g. 192→48 for L1 edge).

```
1 module neuron #(parameter NEURON_WIDTH=8, DATA_BITS=16, WEIGHT_BITS=32,
2   BIAS_BITS=16, WeightFile="...", BiasFile="...")(
3   input clk, rstn, activation_function,
4   input signed [NEURON_WIDTH*DATA_BITS-1:0] data_in_flat,
5   input [31:0] counter,
6   output signed [DATA_BITS+WEIGHT_BITS+8-1:0] data_out);
7
8   localparam ACCUM_BITS = DATA_BITS+WEIGHT_BITS+8, N=4;
9   reg signed [WEIGHT_BITS-1:0] weights [0:NEURON_WIDTH-1];
10  reg signed [BIAS_BITS-1:0] bias_mem [0:0];
11  initial begin
12    $readmemb(WeightFile, weights);
13    $readmemb(BiasFile, bias_mem);
14  end
15
16  // 4 parallel multiply lanes
17  wire signed [DATA_BITS-1:0] x [0:N-1];
18  wire signed [WEIGHT_BITS-1:0] w [0:N-1];
19  reg signed [DATA_BITS+WEIGHT_BITS-1:0] prod [0:N-1];
20  genvar k;
21  generate for (k=0; k<N; k=k+1) begin : lane
22    wire [31:0] idx = (counter>=1) ? (counter-1)*N+k : 0;
23    assign w[k] = (counter>=1 && idx<NEURON_WIDTH) ? weights[idx] : 0;
24    assign x[k] = (counter>=1 && idx<NEURON_WIDTH) ?
25      data_in_flat[idx*DATA_BITS +: DATA_BITS] : 0;
26    always @(posedge clk or negedge rstn)
27      if (!rstn) prod[k] <= 0; else prod[k] <= w[k]*x[k];
28  end endgenerate
29
30  // 2-level adder tree: (p0+p1) + (p2+p3)
31  wire signed [DATA_BITS+WEIGHT_BITS:0] s01, s23;
32  assign s01 = {{1{prod[0][DATA_BITS+WEIGHT_BITS-1]}}, prod[0]}
33    + {{1{prod[1][DATA_BITS+WEIGHT_BITS-1]}}, prod[1]};
34  assign s23 = {{1{prod[2][DATA_BITS+WEIGHT_BITS-1]}}, prod[2]}
35    + {{1{prod[3][DATA_BITS+WEIGHT_BITS-1]}}, prod[3]};
36  wire signed [DATA_BITS+WEIGHT_BITS+1:0] partial = s01 + s23;
37
38  // Accumulator: starts at counter=2 (1cy mult latency)
39  reg signed [ACCUM_BITS-1:0] accumulator, data_out_r;
40  always @(posedge clk or negedge rstn) begin
41    if (!rstn) begin accumulator <= 0; data_out_r <= 0; end
42    else if (counter==1) begin accumulator <= 0; data_out_r <= 0; end
43    else if (counter==2) begin accumulator <= accumulator+partial;
44      data_out_r <= accumulator+partial; end
45    else if (counter>=3) begin accumulator <= accumulator+partial;
46      data_out_r <= accumulator; end
47  end
48  assign data_out = data_out_r;
49 endmodule
```

A.2 ReLU + Saturation (ReLU.v)

Adds bias on the final counter tick, optionally applies ReLU, and saturates the wide accumulator down to DATA_BITS (Q4.4).

```
1 module ReLU #(parameter DATA_BITS=8, WEIGHT_BITS=8, COUNTER_END=6,
2   BIAS_BITS=8, ACCUM_EXTRA=8, WEIGHT_FRAC_BITS=6)(
3   input clk, rstn, activation_function, input [31:0] counter,
4   input signed [DATA_BITS+WEIGHT_BITS+ACCUM_EXTRA-1:0] mult_sum_in,
5   input signed [BIAS_BITS-1:0] b,
6   output reg signed [DATA_BITS-1:0] neuron_out);
7   localparam signed [DATA_BITS-1:0] OUT_MAX = {1'b0, {(DATA_BITS-1){1'b1}}};
8   localparam signed [DATA_BITS-1:0] OUT_MIN = {1'b1, {(DATA_BITS-1){1'b0}}};
9   localparam signed [DATA_BITS+WEIGHT_BITS+ACCUM_EXTRA-1:0]
10     MAX_VAL = {{(WEIGHT_BITS+ACCUM_EXTRA){1'b0}}, OUT_MAX},
11     MIN_VAL = {{(WEIGHT_BITS+ACCUM_EXTRA){1'b1}}, OUT_MIN};
12   reg signed [DATA_BITS+WEIGHT_BITS+ACCUM_EXTRA-1:0] biased_sum;
13   reg signed [DATA_BITS-1:0] neuron_out_next;
14   always @(posedge clk or negedge rstn) begin
15     if (!rstn) neuron_out <= 0;
16     else if (counter == COUNTER_END) begin
17       biased_sum = mult_sum_in +
18         {{(DATA_BITS+WEIGHT_BITS+ACCUM_EXTRA-BIAS_BITS){b[BIAS_BITS-1]}}, b};
```


A.5 Layer Normalization Top-Level FSM (layer_norm.v)

Sequences the four sub-modules implementing $y_i = ((x_i - \mu)/\sqrt{\sigma^2 + \epsilon})\gamma_i + \beta_i$ then optional ReLU. Sub-module latencies: mean 6, var 7, inv_sqrt 4, normalize 3, relu 1.

```
1 localparam [3:0] S_IDLE=0, S_CAPTURE=1, S_WAIT_MEAN=2, S_START_VAR=3,
2   S_WAIT_VAR=4, S_START_SQRT=5, S_WAIT_SQRT=6, S_START_NORM=7,
3   S_WAIT_NORM=8, S_START_RELU=9, S_WAIT_RELU=10, S_OUTPUT=11;
4 always @(*) begin
5   next_state = state;
6   case (state)
7     S_IDLE      : if (valid_in) next_state = S_CAPTURE;
8     S_CAPTURE   : next_state = S_WAIT_MEAN;
9     S_WAIT_MEAN : if (mean_valid) next_state = S_START_VAR;
10    S_START_VAR  : next_state = S_WAIT_VAR;
11    S_WAIT_VAR   : if (var_valid) next_state = S_START_SQRT;
12    S_START_SQRT : next_state = S_WAIT_SQRT;
13    S_WAIT_SQRT  : if (sqrt_valid) next_state = S_START_NORM;
14    S_START_NORM : next_state = S_WAIT_NORM;
15    S_WAIT_NORM  : if (norm_valid) next_state = S_START_RELU;
16    S_START_RELU : next_state = S_WAIT_RELU;
17    S_WAIT_RELU  : if (relu_valid) next_state = S_OUTPUT;
18    S_OUTPUT     : next_state = S_IDLE;
19  endcase
20 end
```

A.6 Counter & Multiplier Primitives

```
1 module counter #(parameter END_COUNTER = 32'd0)(
2   input clk, rstn, output reg [31:0] counter_out,
3   output reg counter_donestatus);
4   always @(posedge clk) begin
5     if (!rstn) begin counter_out<=0; counter_donestatus<=0; end
6     else if (counter_out >= END_COUNTER+1) begin
7       counter_out <= END_COUNTER+1; counter_donestatus <= 1'b1;
8     end else begin
9       counter_out <= counter_out + 1'b1; counter_donestatus <= 1'b0;
10    end
11  end
12 endmodule
13
14 module multiplier #(parameter DATA_BITS=8, WEIGHT_BITS=8)(
15   input clk, rstn,
16   input signed [WEIGHT_BITS-1:0] w,
17   input signed [DATA_BITS-1:0] x,
18   output reg signed [DATA_BITS+WEIGHT_BITS-1:0] mult_result);
19   always @(posedge clk or negedge rstn)
20     if (!rstn) mult_result <= 0;
21     else      mult_result <= w * x;
22 endmodule
```